

Renewind company project

Model Tuning

Pauline Zeestraten

Date: March 8, 2024

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model performance summary for hyperparameter tuning.
- Model building with pipeline
- Appendix

Executive Summary

- We were limited in our ability to gain insight into the large number of predictor variables due to the anonymous nature of sensor data collection. It would greatly benefit Renewind company if it could at the very minimum, differentiate the nature of the current predictor variables between environmental factors on one hand, such as temperature, humidity, wind speed, etc, and various parts of the wind turbine on the other, (gearbox, tower, blades, break, etc). Renewind has control over the latter set of predictor variables, but not over the first. To simplify the model, and to make its prediction ability more robust, it would be helpful to drop irrelevant features.
- For instance, the feature importance of our best model showed that variable V18 had by far the greatest influence on the target variable indicating failure. Unfortunately, we have no further information with regards to the exact nature of the sensor data of variable V18.

Business Problem Overview

Wind energy, one of the most developed renewable energy alternative technologies worldwide, plays an increasingly important role in the global energy mix. The U.S Department of Energy has put together a guide to achieving operational efficiency using predictive maintenance practices.

Predictive maintenance uses sensor information and analysis methods to measure and predict degradation and future component capability. The idea behind predictive maintenance is that failure patterns are predictable and if component failure can be predicted accurately and the component is replaced before it fails, the costs of operation and maintenance will be much lower.

The sensors fitted across different machines involved in the process of energy generation collect data related to various environmental factors (temperature, humidity, wind speed, etc.) and additional features related to various parts of the wind turbine (gearbox, tower, blades, break, etc).

“ReneWind” is a company working on improving the machinery and processes involved in the production of wind energy using machine learning and has collected data of generator failure of wind turbines using sensors.

Business Solution Approach

Renewind has shared a ciphered version of the data, as the data collected through sensors is confidential. The dataset has 40 predictors, 20,000 observations in the training set, and 5,000 in the test set.

Our approach is to build various classification models, tune them, and find the best one that will help identify failures so that the generators can be repaired before failing/breaking to reduce the overall maintenance cost. The nature of predictions made by the classification model will translate as follows:

- True positives (TP) are failures correctly predicted by the model. These will result in repair costs.
- False negatives (FN) are real failures where there is no detection by the model. These will result in replacement costs.
- False positives (FP) are detections where there is no failure. These will result in inspection costs.

It is given that the cost of repairing a generator is much less than the cost of replacing it, and the cost of the inspection is less than the cost of repair.

“1” in the target variables should be considered as “failure” and “0” represents “No failure”.

EDA Results

- We worked with a training dataset of 20,000 rows and 41 columns, and a test dataset of 5,000 rows and 41 columns.
- All columns are numerical in nature, and divided into one target variable and 40 predictor variables.
- The target variable differentiates 1 as failure, and 0 as no failure. In the training set we see 1110 entries classified as 1 (failure), and 18890 as 0 (non-failure). For the test set this is 282 (failure) and 4718 (non-failure). The dataset is therefore imbalanced.
- The data contains negative values that were not treated as these are legitimate values recorded by sensors. We were limited in our ability to gain insight with regards to the 40 predictor variables, due to its ciphered, anonymous nature of the data recorded by sensors.
- We observe that the majority of the independent variables have fairly normal distributions with long tails on both ends, meaning outliers are common in this dataset.
- A handful of variables are right skewed.

Note: *You can use more than one slide if needed*

[Link to Appendix slide on data background check](#)

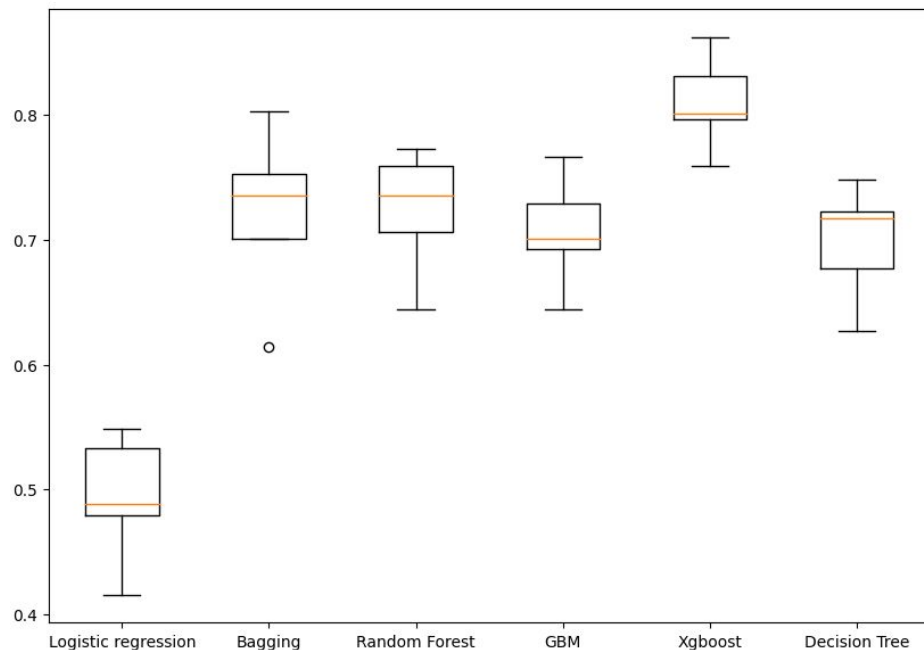
Data Preprocessing

- Duplicate value check: No duplicate values were found
- Missing value treatment: We observed 18 missing values each in the first two variables of the training dataset, and 5 and 6 values respectively in the corresponding predictor variables in the test set. We remedied these by imputing the mean of the said variables.
- Outlier check (treatment if needed): Outliers are the default for the predictor variables in this dataset (with the exception of V40). The majority of the predictor variables also show normal distributions suggesting that these long tails are not errors and should therefore be taken into consideration in building a predictive model.
- Feature engineering:
- Data preparation for modeling

Note: You can use more than one slide if needed

Model Performance Summary (on original data)

Algorithm Comparison



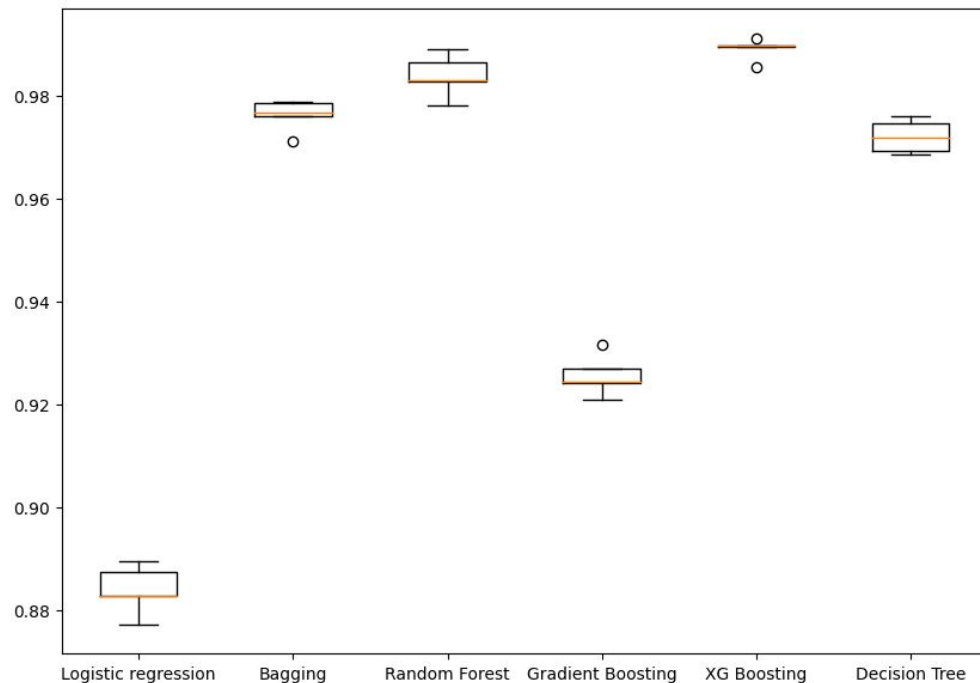
Boxplot for cross validation scores of six different algorithm models on original training data.

We observe that Xgboost, followed by Random Forest, and Bagging perform best.

[Link to Appendix slide on model assumptions](#)

Model Performance Summary (oversampled data)

Algorithm Comparison of models built on oversampled data

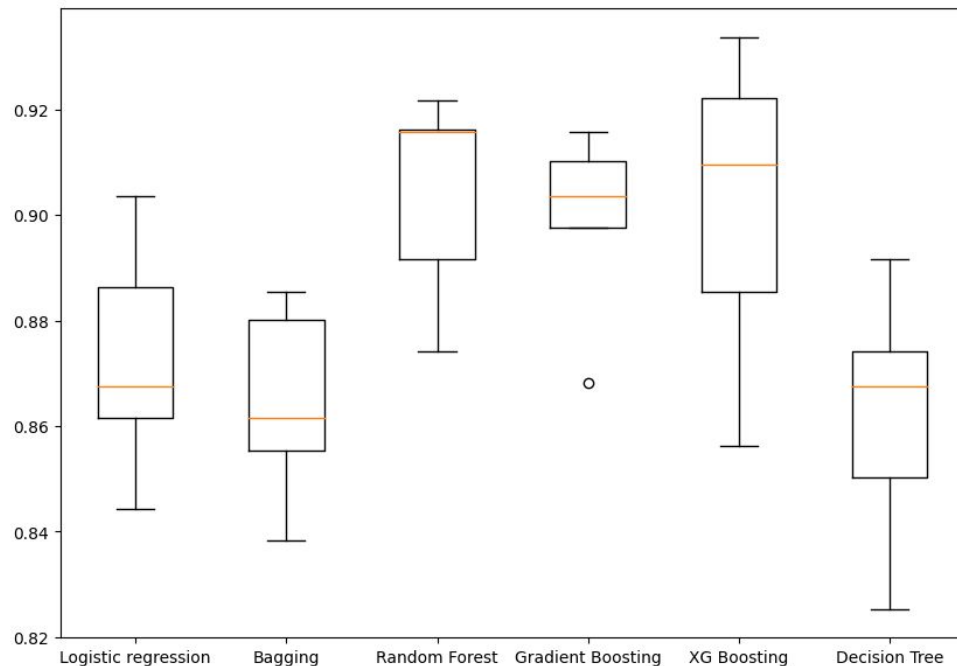


We observe that the individual range of CV scores is much narrower in the algorithm models built on oversampled data.

The CV scores are also very high, suggesting overfitting the training data, and poor generalizing abilities on test set.

Model Performance Summary (undersampled data)

Algorithm Comparison of models built on undersampled data



We observe that a wider range of CV scores of the various models.

XG Boosting is here also the best scoring model, followed by Random Forest.

Model Performance Summary after Hypertuning

Training performance comparison:

	Gradient Boosting tuned with oversampled data	AdaBoost classifier tuned with oversampled data	Random forest tuned with undersampled data	XGBoost tuned with oversampled data
Accuracy	0.993	0.992	0.961	0.994
Recall	0.992	0.988	0.933	1.000
Precision	0.994	0.995	0.989	0.988
F1	0.993	0.992	0.960	0.994

Validation performance comparison:

	Gradient Boosting tuned with oversampled data	AdaBoost classifier tuned with oversampled data	Random forest tuned with undersampled data	XGBoost tuned with oversampled data
Accuracy	0.969	0.979	0.938	0.967
Recall	0.856	0.856	0.885	0.892
Precision	0.678	0.791	0.468	0.648
F1	0.757	0.822	0.612	0.750

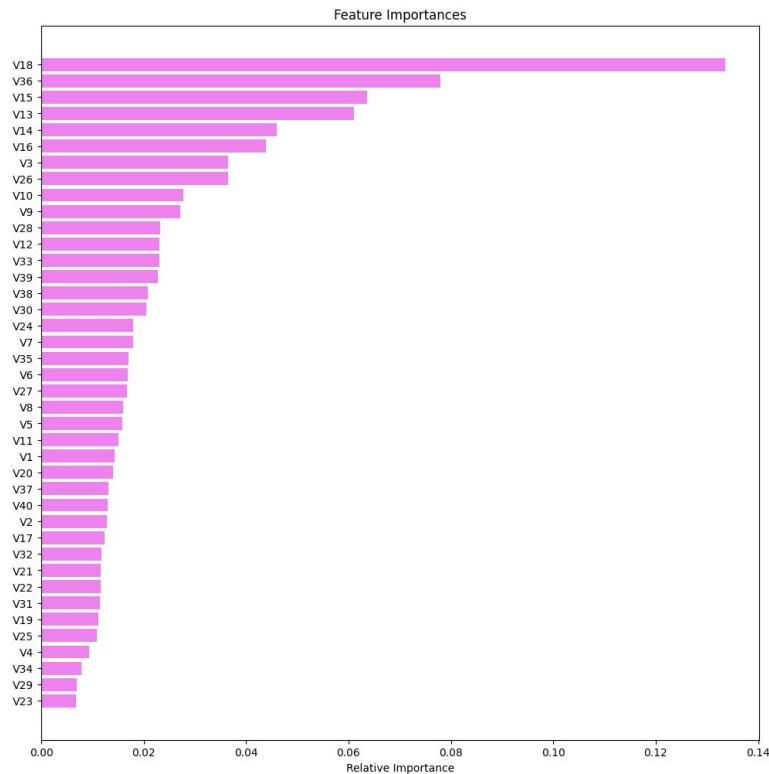
```
xgb2_test_perf = model_performance_classification_sklearn(xgb2, X_test, y_test)
```

xgb2_test_perf

	Accuracy	Recall	Precision	F1
0	1.000	1.000	1.000	1.000

- We observe that for the metric “Recall” which is of most interest for our client Renewind, the highest score in the validation set is delivered by the model “XGBoost tuned with oversampled data”.
- All models score similarly high in the recall metric, and either one would be acceptable as well.
- We proceeded with the XGBoost tuned model on the test set.

Feature Importances of best model with highest Recall score.



- “XGBoost tuned with oversampled data”.

Productionize and test the final model using pipelines

```
[ ] Pipeline_model = Pipeline(steps=[('XGB', xgb2)]) ## Complete the code to create pipeline for

[ ] # Separating target variable and other variables
X1 = data.drop(columns="Target")
Y1 = data["Target"]

# Since we already have a separate test set, we don't need to divide data into train and test

X_test1 = df_test.drop(columns="Target") ## Complete the code to drop target variable from te
y_test1 = df_test["Target"] ## Complete the code to store target variable in y_test1

# We can't oversample/undersample data without doing missing value treatment, so let's first t
imputer = SimpleImputer(strategy="median")
X1 = imputer.fit_transform(X1)

# We don't need to impute missing values in test set as it will be done inside pipeline
```

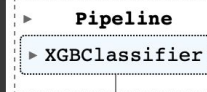
Productionize and test the final model using pipelines

```
#code for oversampling on the data
# Synthetic Minority Over Sampling Technique
sm = SMOTE(sampling_strategy=1, k_neighbors=5, random_state=1)
X_over1, y_over1 = sm.fit_resample(X1, Y1)
```

If the best model is built on the undersampled data, uncomment and run the below code to perform undersampling

```
[ ] ## code for undersampling on the data
## Under Sampling Technique
# rus = RandomUnderSampler(random_state=1, sampling_strategy=1)
# X_train_un, y_train_un = rus.fit_resample(X_train, y_train)
```

```
[ ] Pipeline_model.fit(X_over1, y_over1) ## Complete the code to fit the Model obtained from above
```



```
Pipeline_model_test = model_performance_classification_sklearn(xgb2,X_test, y_test) ## Complete
Pipeline_model_test
```

	Accuracy	Recall	Precision	F1
0	0.958	0.858	0.585	0.695

Productionize and test the final model using pipelines

- Please mention steps taken to create a pipeline for the final model
- Summary of the performance of the model built with pipeline on test dataset
- Summary of most important factors used by the model built with pipeline for prediction

Note: *You can use more than one slide if needed*

[Link to Appendix slide on model assumptions](#)

APPENDIX

EDA code

```
# Checking the number of rows and columns in the training data  
df.shape ## Complete the code to view dimensions of the train data
```

```
# Checking the number of rows and columns in the test data  
df_test.shape ## Complete the code to view dimensions of the test data
```

```
# let's view the first 5 rows of the data  
data.head() ## Complete the code to view top 5 rows of the data
```

```
# let's view the last 5 rows of the data  
data_test.tail() ## Complete the code to view last 5 rows of the data
```

```
# let's check for duplicate values in the data  
data.duplicated().sum() ## Complete the code to check duplicate entries in the data
```

```
# let's check for missing values in the data  
data.isnull().sum() ## Complete the code to check missing entries in the train data
```

EDA code continued

```
# let's check for missing values in the data
```

```
data_test.isnull().sum() ## Complete the code to check missing entries in the test data
```

```
# let's view the statistical summary of the numerical columns in the data
```

```
data.describe() ## Complete the code to print the statistical summary of the train data
```

```
%time ##### Attempt to look at pair plots to detect any correlation, and whether over/undersampling  
would be useful or not.
```

```
sns.pairplot(data, hue='Target', diag_kind='hist')
```

```
CPU times: user 1h 5min 15s, sys: 1min 26s, total: 1h 6min 42s
```

```
Wall time: 1h 8min 51s
```

```
<seaborn.axisgrid.PairGrid at 0x7d3aee8521d0>
```

```
data["Target"].value_counts() ## Complete the code to check the class distribution in target variable for  
train data
```

```
data_test["Target"].value_counts() ## Complete the code to check the class distribution in target  
variable for test data
```

Data pre-processing

```
# Splitting train dataset into training and validation set
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.25, random_state=1, stratify=y) ##
```

```
Complete the code to split the train dataset into train test in the ratio 75:25. ### Stratify = to retain  
the same proportion of y in both train and validation set.
```

```
# Checking the number of rows and columns in the X_train data
```

```
X_train.shape ## Complete the code to view dimensions of the X_train data  
(15000, 40)
```

```
# Checking the number of rows and columns in the X_val data
```

```
X_val.shape ## Complete the code to view dimensions of the X_val data  
(5000, 40)
```

```
# Dividing test data into X_test and y_test
```

```
X_test = data_test.drop(["Target"], axis=1) ## Complete the code to drop target variable from test data
```

```
y_test = data_test["Target"] ## Complete the code to store target variable in y_test
```

Data pre-processing continued

```
# Checking the number of rows and columns in the X_test data
```

```
X_test.shape ## Complete the code to view dimensions of the X_test data  
(5000, 40)
```

```
# Transform the validation data
```

```
X_val = pd.DataFrame(imputer.fit_transform(X_val), columns=X_train.columns) ## Complete the code to  
impute missing values in X_val without data leakage
```

```
# Transform the test data
```

```
X_test = pd.DataFrame(imputer.fit_transform(X_test), columns=X_train.columns) ## Complete the code to  
impute missing values in X_test without data leakage
```

```
X_val.isna().sum() ## Complete the code to check the count of missing values in validation set
```

```
X_test.isna().sum() ## Complete the code to check the count of missing values in test set
```

Model Building (original data)

```
models = [] # Empty list to store all the models

# Appending models into the list
models.append(("Logistic regression", LogisticRegression(random_state=1)))
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random Forest", RandomForestClassifier(random_state=1)))
models.append(("GBM", GradientBoostingClassifier(random_state=1)))
models.append(("Xgboost", XGBClassifier(random_state=1)))
models.append(("Decision Tree", DecisionTreeClassifier(random_state=1)))
## Complete the code to append remaining 4 models in the list models
```

Model Performance Summary (original data)



Cross-Validation performance on training dataset:

```
Logistic regression: 0.4927566553639709
Bagging: 0.7210807301060529
Random Forest: 0.7235192266070268
GBM: 0.7066661857008874
Xgboost: 0.8100497799581561
Decision Tree: 0.6982829521679532
```

Validation Performance:

```
Logistic regression: 0.48201438848920863
Bagging: 0.7302158273381295
Random Forest: 0.7266187050359713
GBM: 0.7230215827338129
Xgboost: 0.8309352517985612
Decision Tree: 0.7050359712230215
CPU times: user 5min 8s, sys: 683 ms, total: 5min 9s
Wall time: 5min 25s
```

We observe this model on original data shows no overfitting, as the difference between mean cross validation recall score performance on training set and validation performance is very minimal. Let's see if we can get higher scores using over- and under sampling of training data.

Model Building (oversampled data using SMOTE)

```
# Synthetic Minority Over Sampling Technique
```

```
sm = SMOTE(sampling_strategy=1, k_neighbors=5, random_state=1)
```

```
X_train_over, y_train_over = sm.fit_resample(X_train, y_train)
```

```
models = [] # Empty list to store all the models # Appending models into the list
```

```
models.append(("Logistic regression", LogisticRegression(random_state=1)))
```

```
models.append(("Bagging", BaggingClassifier(random_state=1)))
```

```
models.append(("Random Forest", RandomForestClassifier(random_state=1)))
```

```
models.append(("Gradient Boosting", GradientBoostingClassifier(random_state=1)))
```

```
models.append(("XG Boosting", XGBClassifier(random_state=1)))
```

```
models.append(("Decision Tree", DecisionTreeClassifier(random_state=1)))
```

```
## Complete the code to append remaining 4 models in the list models
```

```
cv_result = cross_val_score(estimator=model, X=X_train_over, y=y_train_over, scoring=scorer, cv=kfold)
```

```
## Complete the code to build models on oversampled data
```

```
for name, model in models:
```

```
model.fit(X_train_over, y_train_over) ## Complete the code to build models on oversampled data
```

Model Performance Summary (oversampled data)

Cross-Validation performance on training dataset:

Logistic regression: 0.883963699328486

Bagging: 0.9762141471581656

Random Forest: 0.9839075260047615

Gradient Boosting: 0.9256068151319724

XG Boosting: 0.9891305241357218

Decision Tree: 0.9720494245534969

Validation Performance:

Logistic regression: 0.8489208633093526

Bagging: 0.8345323741007195

Random Forest: 0.8489208633093526

Gradient Boosting: 0.8776978417266187

XG Boosting: 0.8669064748201439

Decision Tree: 0.7769784172661871

CPU times: user 8min 43s, sys: 1.75 s, total: 8min 45s

Wall time: 8min 59s

Performance of this model on training set varies between 0.88 to 0.98, which is an improvement from the model built on the original dataset. Performance on the training set improved but the model is not able to replicate the same for the validation set, which indicates that the model is overfitting. Let's try regularization to see if overfitting can be reduced.

Model Building (undersampled data)

```
# Using RandomUnderSampler technique
```

```
rus = RandomUnderSampler(random_state=1, sampling_strategy=1)
```

```
X_train_un, y_train_un = rus.fit_resample(X_train, y_train)
```

```
# Appending models into the list
```

```
models.append(("Logistic regression", LogisticRegression(random_state=1)))
```

```
models.append(("Bagging", BaggingClassifier(random_state=1)))
```

```
models.append(("Random Forest", RandomForestClassifier(random_state=1)))
```

```
models.append(("Gradient Boosting", GradientBoostingClassifier(random_state=1)))
```

```
models.append(("XG Boosting", XGBClassifier(random_state=1)))
```

```
models.append(("Decision Tree", DecisionTreeClassifier(random_state=1)))
```

```
## Complete the code to append remaining 4 models in the list models
```

```
cv_result = cross_val_score(estimator=model, X=X_train_un, y=y_train_un, scoring=scorer, cv=kfold)
```

```
## Complete the code to build models on undersampled data
```

```
for name, model in models: model.fit(X_train_un, y_train_un)
```

```
## Complete the code to build models on undersampled data
```

[Link to Appendix slide on model assumptions](#)

Model Performance Summary (undersampled data)



Cross-Validation performance on training dataset:

```
Logistic regression: 0.8726138085275232
Bagging: 0.8641945025611427
Random Forest: 0.9038669648654498
Gradient Boosting: 0.8990621167303946
XG Boosting: 0.9014717552846114
Decision Tree: 0.8617776495202367
```

Validation Performance:

```
Logistic regression: 0.8525179856115108
Bagging: 0.8705035971223022
Random Forest: 0.8920863309352518
Gradient Boosting: 0.8884892086330936
XG Boosting: 0.89568345323741
Decision Tree: 0.841726618705036
CPU times: user 30.2 s, sys: 438 ms, total: 30.7 s
Wall time: 31.1 s
```

Performance of this model trained on data with undersampling varies between 0.86 to 0.90, while validation performance ranges between 0.84 and 0.89. This means this model is not over- nor underfitted. The overall recall score performance among the various algorithm classifiers is quite good.

Slide Header

- Please add any other pointers (if needed)



Happy Learning !

